# *STRIPE* PROJECT

# 8. Global Data Architecture

## D1. Data Architecture Diagrams

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / D1 : Data Architecture Diagrams / v33 - 2026/11/05.*

---


- D1.1. Contexte
- D1.2. Vue d'ensemble
- D1.3. Vue du coeur métier
  - D1.3.1. Architecture
  - D1.3.2. Flux de données
  - D1.3.3. Modèles de données
- D1.4. Vue du scoring de fraude
  - D1.4.1. Architecture
  - D1.4.2. Flux de données
  - D1.4.3. Modèles de données
- D1.5. Vue du reporting de sécurité et de conformité
  - D1.5.1. Architecture
  - D1.5.2. Flux de données
  - D1.5.3. Modèles de données
- D1.6. Technologies
- D1.7. Remarques
- Références


### D1.1. Contexte

Ce document présente l'architecture du système de données du projet *Stripe* [D0]. Il décrit l'intégration des systèmes *OLTP* [D2], *OLAP* [D3] et *NoSQL* [D4], et détaille les flux de données et les pipelines qui lient ces systèmes.

**Points particulier de l'architecture**

L'architecture est structurée pour permettre au chemin transactionnel de ne jamais être ralenti par les processus de sécurité, de conformité, d'analytique ou de détection de fraude.*

Les flux de données qui partent du système transactionnel vers les autres systèmes de données sont donc toujours *asynchrones*. En particulier, la réplication des changements d'état transactionnels ne doit pas être synchrone : le système bannit donc les déclencheurs synchrones sur les changements d'état (*triggers*) et privilégie l'exploitation asynchrone du flux *CDC* natif du système *OLTP*.

Les détails d'implémentation de l'intégration OLTP/OLAP/NoSQL sont détaillés dans [D5].

L'intégration des questions de sécurité et de conformité sont détaillés dans [D6].

L'intégration des données d'entrainement des modèles d'apprentissage automatique, ainsi que les processus d'entrainement, d'évaluation, de déploiement, de monitoring et de réentrainement sont détaillés dans [D7].

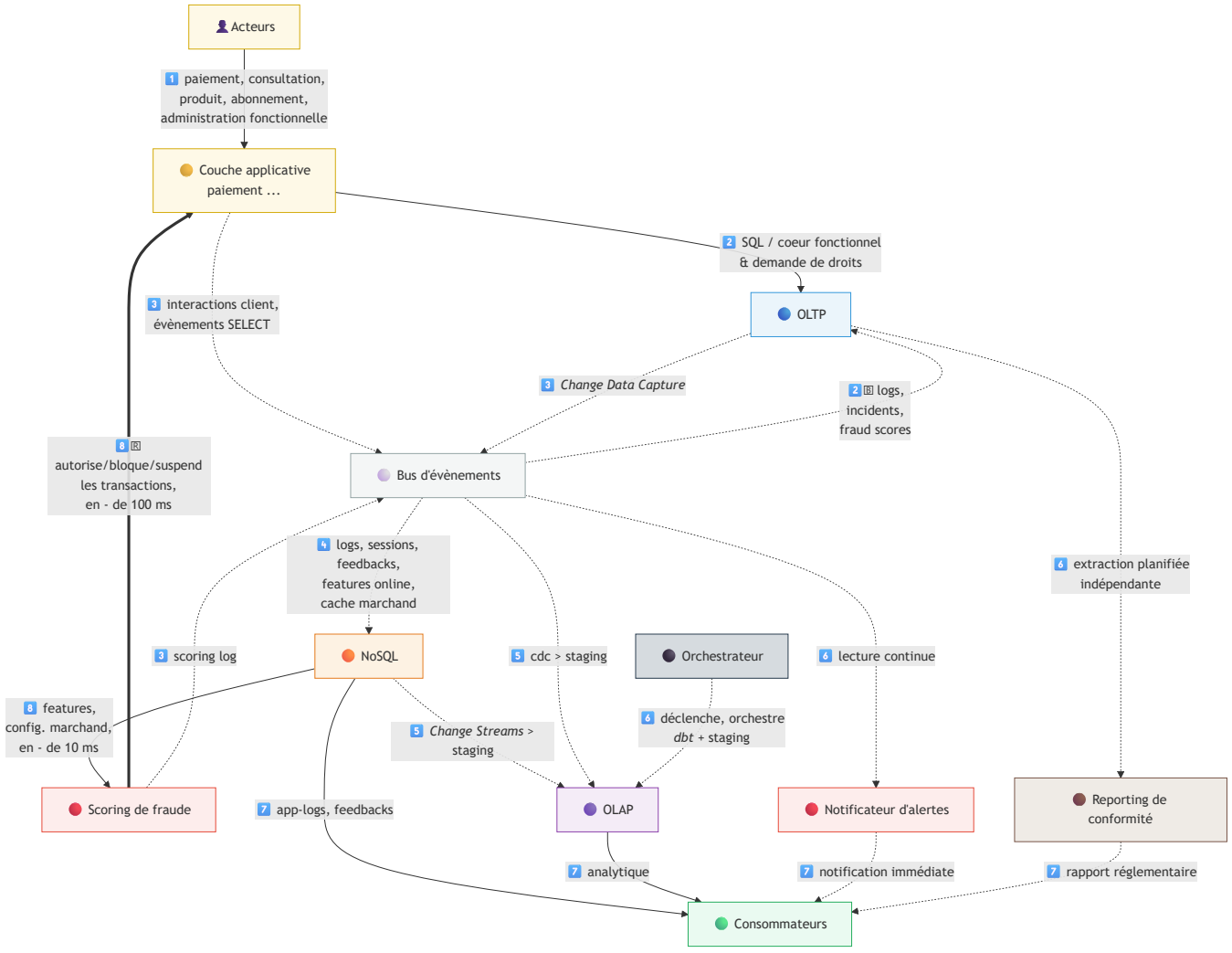
### D1.2. Vue d'ensemble

Le diagramme suivant montre l'architecture générale du système. Le système est basé sur x sous-systèmes :

- une couche applicative interne non explicitée qui centralise les flux d'entrée ;
- le système OLTP ;
- le service de scoring de fraude en temps réel ;
- un bus d'évènements en flux continu pour redistribuer les évènements applicatifs et transactionnels de manière asynchrone ;
- le système de données NoSQL, basé sur la notion de document (des "collections") ;
- un orchestrateur pour planifier et exécuter l'extraction des données NoSQL vers OLAP ;
- les sous-systèmes finaux qui préparent et servent les données aux consommateurs finaux :
  - le système OLAP pour les données analytiques
  - le système d'alerte de sécurité en temps réel,

- le système de reporting de conformité pour les données transactionnelles,
- un accès direct à certaines collections NoSQL : logs d'applications, feedbacks des clients.

Les flux de données sont détaillés dans §D1.3.2.

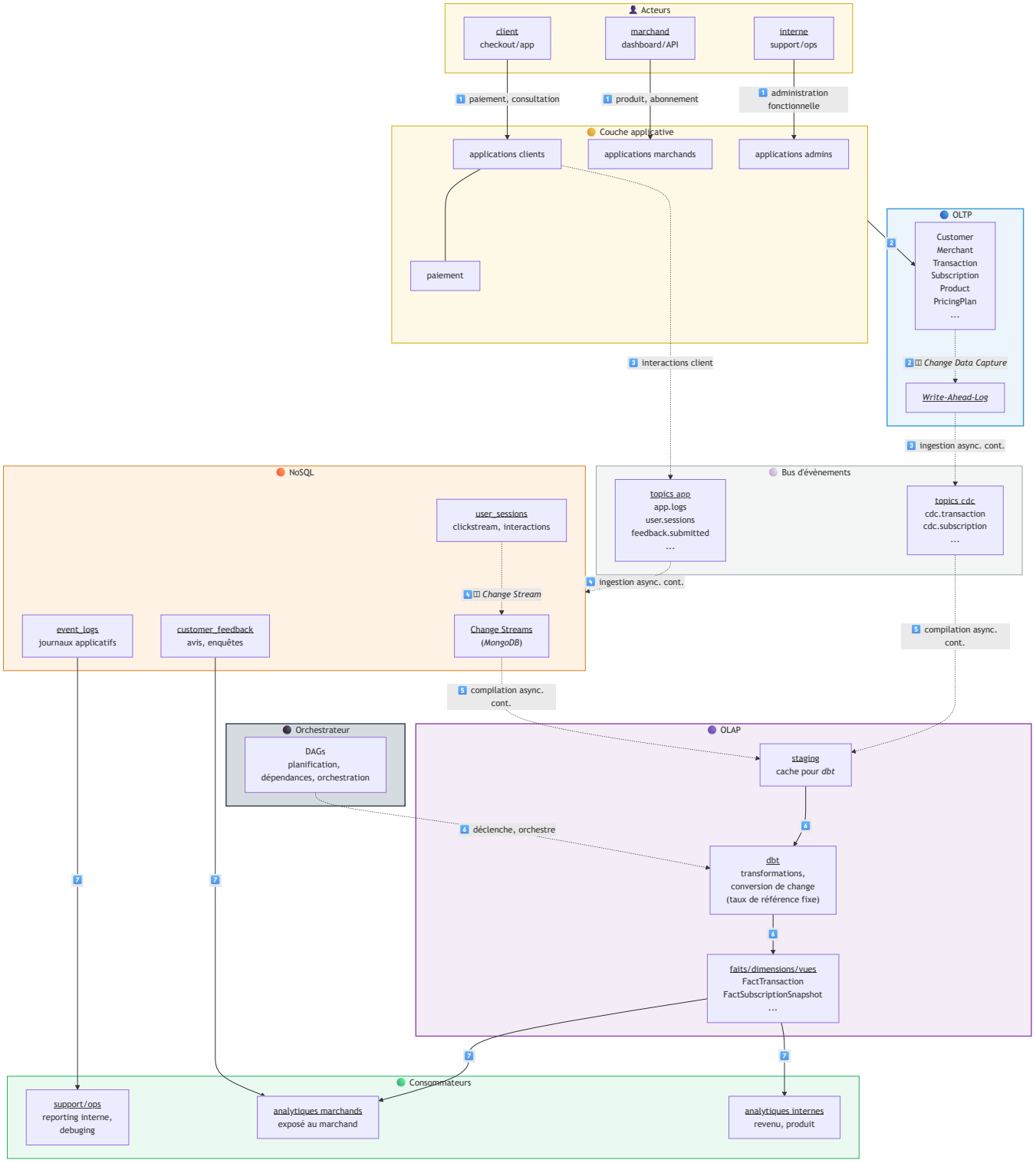



## D1.3. Vue du coeur métier

### D1.3.1. Architecture

Le coeur métier du système est centré sur la gestion des données applicatives : transactions, marchands, clients et entités associées. La vue suivante exclue donc les questions de sécurité et de conformité, de détection de fraude en temps réel, ainsi que l'intégration des modèles d'apprentissage automatique et des services qui en découlent.

Les flux de données sont détaillés dans §D1.3.2.




### D1.3.2. Flux de données

Explication des flux de données du coeur de métier :

| # | Flux | Mécanisme | Justification |
|---|---|---|---|
| 1 | Acteurs → Apps | écr. sync. directe | Flux d'entrée des données dans le système ; Passe par une couche applicative interne qui isole le système des acteurs externes : client/marchand/interne. |
| 2 | Apps → Core OLTP | écr. sync. directe | Logique de traitement (validation, orchestration du paiement) séparée de la logique de persistance. |
| 3 | Core OLTP → Topic `cdc` | lect. async. cont. du *WAL* | Journalisation des modifications des tables Core ( `Transaction` , etc.) => banir les déclencheurs synchrones (*triggers*) => lecture *asynchrone* du journal système (WAL : Write-Ahead-Log) + publication des modifications dans les topics `cdc` . |

| # | Flux | Mécanisme | Justification |
|---|---|---|---|
| 4 | Topic `app` → NoSQL | ingest. async. cont. | Transport des logs d'application, clickstreams, interactions avec l'utilisateur, vers NoSQL. |
| 5 | Topic `cdc` → staging OLAP | extract. async. cont. | Chargement temporaire des modifications des tables Core (via les topics `cdc` ) dans l'entrepôt OLAP |
| 5 | NoSQL *Change Streams* → staging OLAP | extract. async. cont. | Chargement temporaire des modifications de la collection NoSQL `user_sessions` dans l'entrepôt OLAP |
| 6 | dbt + staging → OLAP Core | transf. plan. | L'orchestrateur déclenche les processus de transformation (ici, portés par *dbt*), les processus transforment et chargent directement dans l'entrepôt OLAP ; |
| 7 | OLAP Core → analytiques | req. analytiques | 1 consommateur externe : marchand, 1 consommateur interne : analyste revenus & produits |
| 7 | NoSQL `customer_feedback` → marchand | req. analytiques | 1 consommateur externe : marchand |
| 7 | NoSQL `event_logs` → admins | req. analytiques | 1 consommateur interne : administrateur fonctionnel |

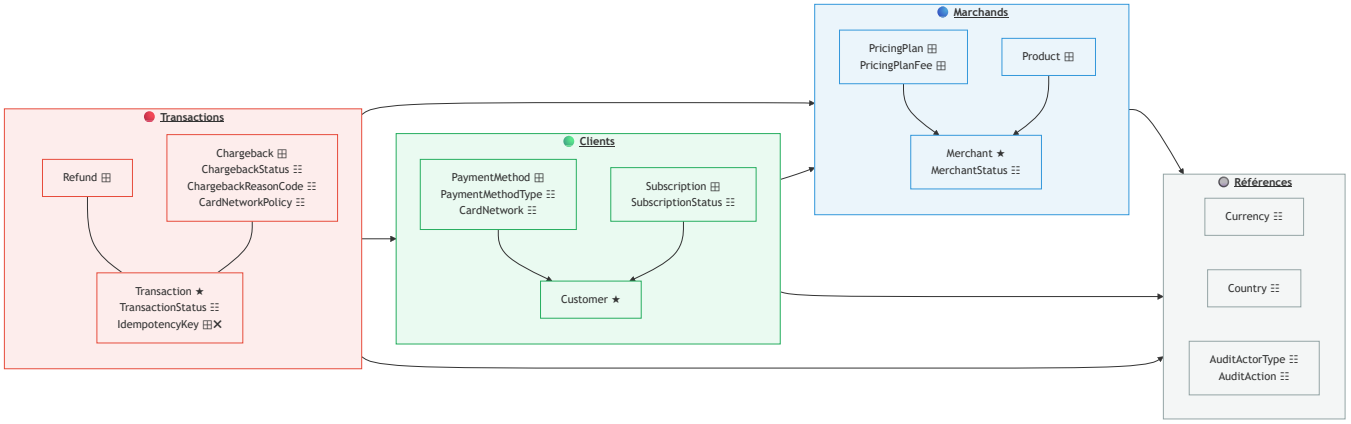
> *abréviations :*
>
> - écr. = écriture
> - lect. = lecture
> - cont. = continue
> - sync. = synchrone
> - async. = asynchrone
> - ingest. = ingestion
> - compil. = compilation
> - extract. = extraction
> - transf. = transformation
> - plan. = planifié
> - req. = requêtes

## D1.3.3. Modèles de données

### D1.3.3.1. Données transactionnelles

Le diagramme suivant montre une vue compacte du modèle transactionnel logique lié au coeur de métier. Les entités principales du coeur de métier sont les *transactions*, les *marchands* et les *clients*. Les liens sont des liens de *dépendance*, au sens suivant : si le groupe *A* pointe sur le groupe *B*, alors des tables de *A* référence des tables de *B*.

Les détails du modèle transactionnel logique sont dans [D2] (cf. §D2.4 : *Vue Marchands*, §D2.5 : *Vue Clients*, §D2.6 : *Vue Transactions*)
.

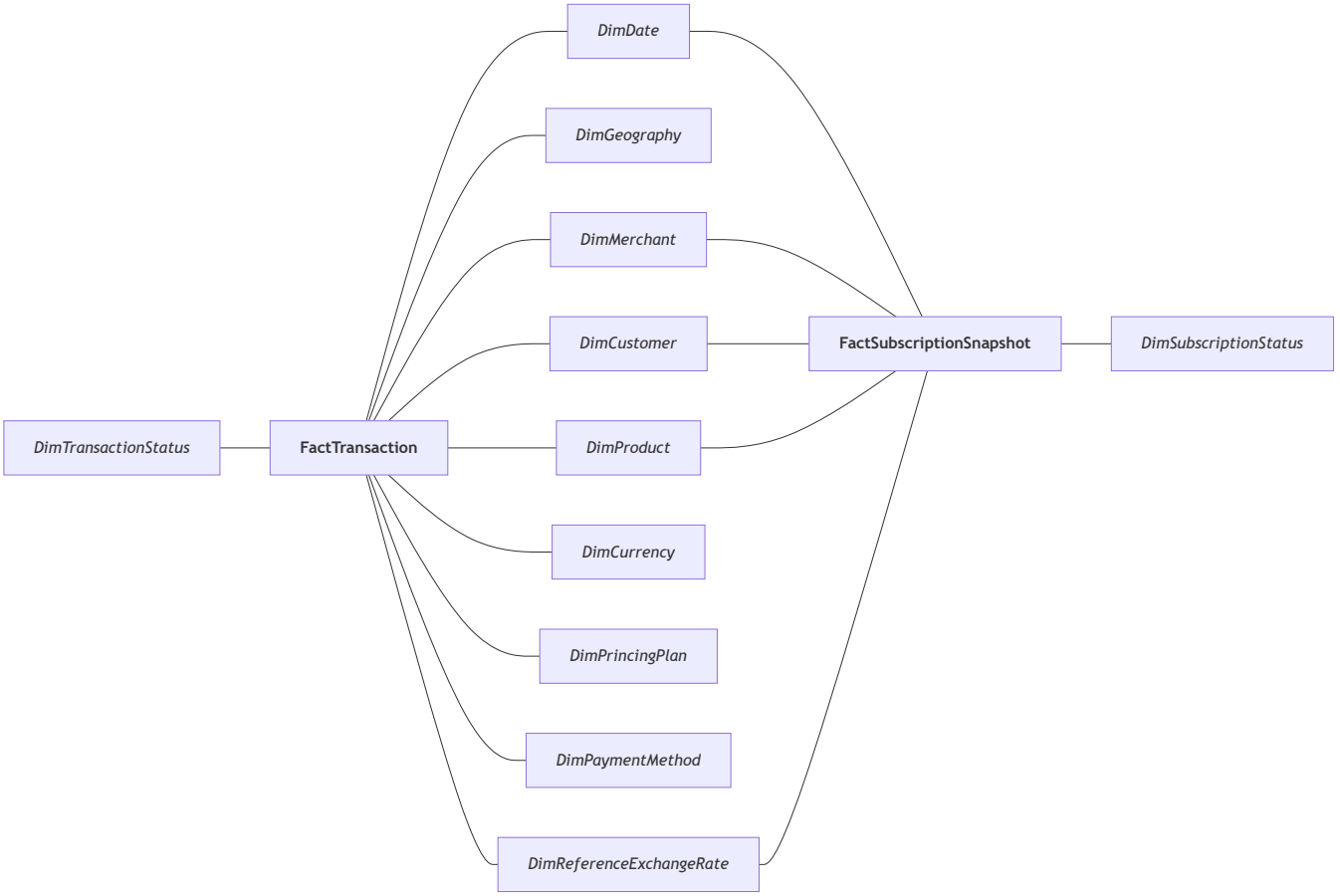



> *Etiquettes de table :*
>
> - ★ : Entité métier principale ;
> - ⊞ : Entité métier secondaire (extension directe ou indirecte d'une entité principale) ;
> - ☷ : Enumération ;
> - ▤ : Journal ;
> - ✕ : ne peut pas être journalisé (dans les journaux des accès et des écritures) ;

#### D1.3.3.2. Données analytiques

Le diagramme suivant montre une vue compacte du modèle analytique logique lié au coeur de métier. Les processus analytiques du coeur de métier sont les *revenus* et les *abonnements*.

Les détails du modèle analytique logique sont dans [D3] (cf. §D3.3.1 : *Revenus*, §D3.3.3 : *Abonnements*).

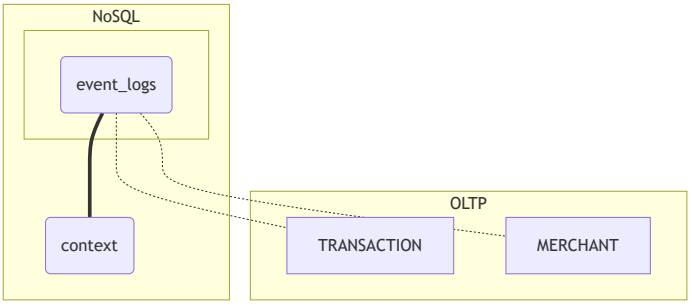



#### D1.3.3.3. Données non-structurées

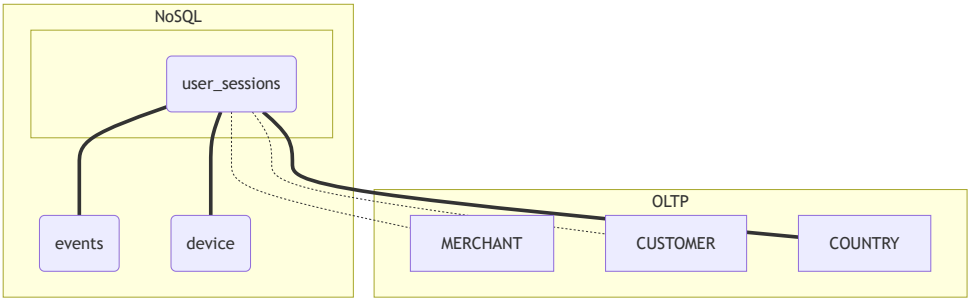
Le diagramme suivant montre une vue compacte des collections non structurées liées au coeur de métier. Les collections du coeur de métier sont les *journaux d'application* (*event-logs*), les *sessions d'utilisateur* (*user-session*) et les *retours clients* (*customer-feedbacks*).

Les détails du modèle des données non-structurées sont dans [D4] (cf. §D4.5.2 : *event-logs*, §D4.5.3 : *user-sessions*, §D4.5.5 :*customer-feedbacks*).

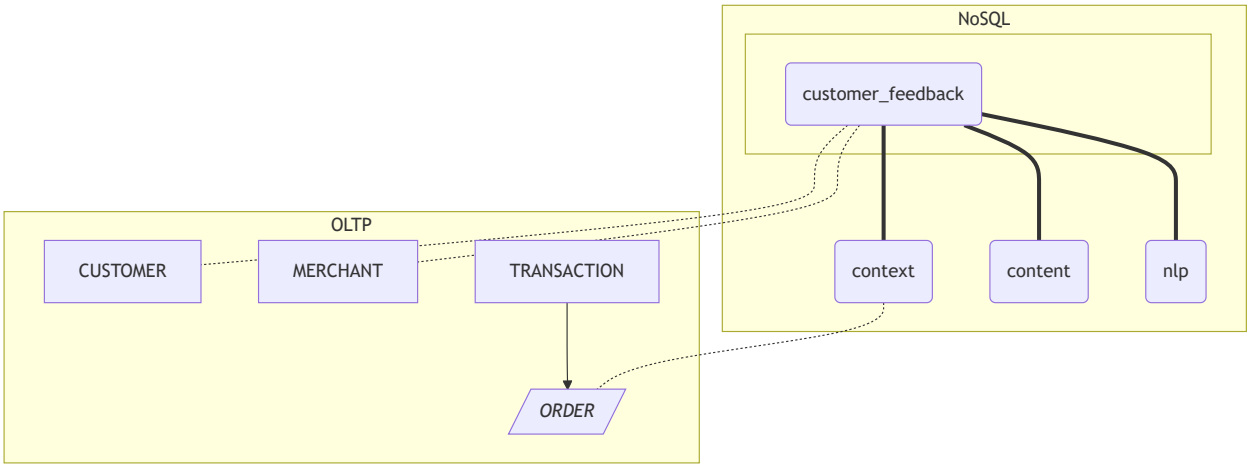
**event-logs**




**user-sessions**




**customer-feedbacks**




## D1.4. Vue du scoring de fraude

### D1.4.1. Architecture

Le diagramme suivant montre l'architecture du système de détection de fraude et les flux de données associés. L'intégration des processus d'entrainement du modèles de scoring et la gestion des données d'entrainement associées font l'objet d'un livrable dédiée [D7].

Les flux de données sont détaillés dans §D1.4.2.

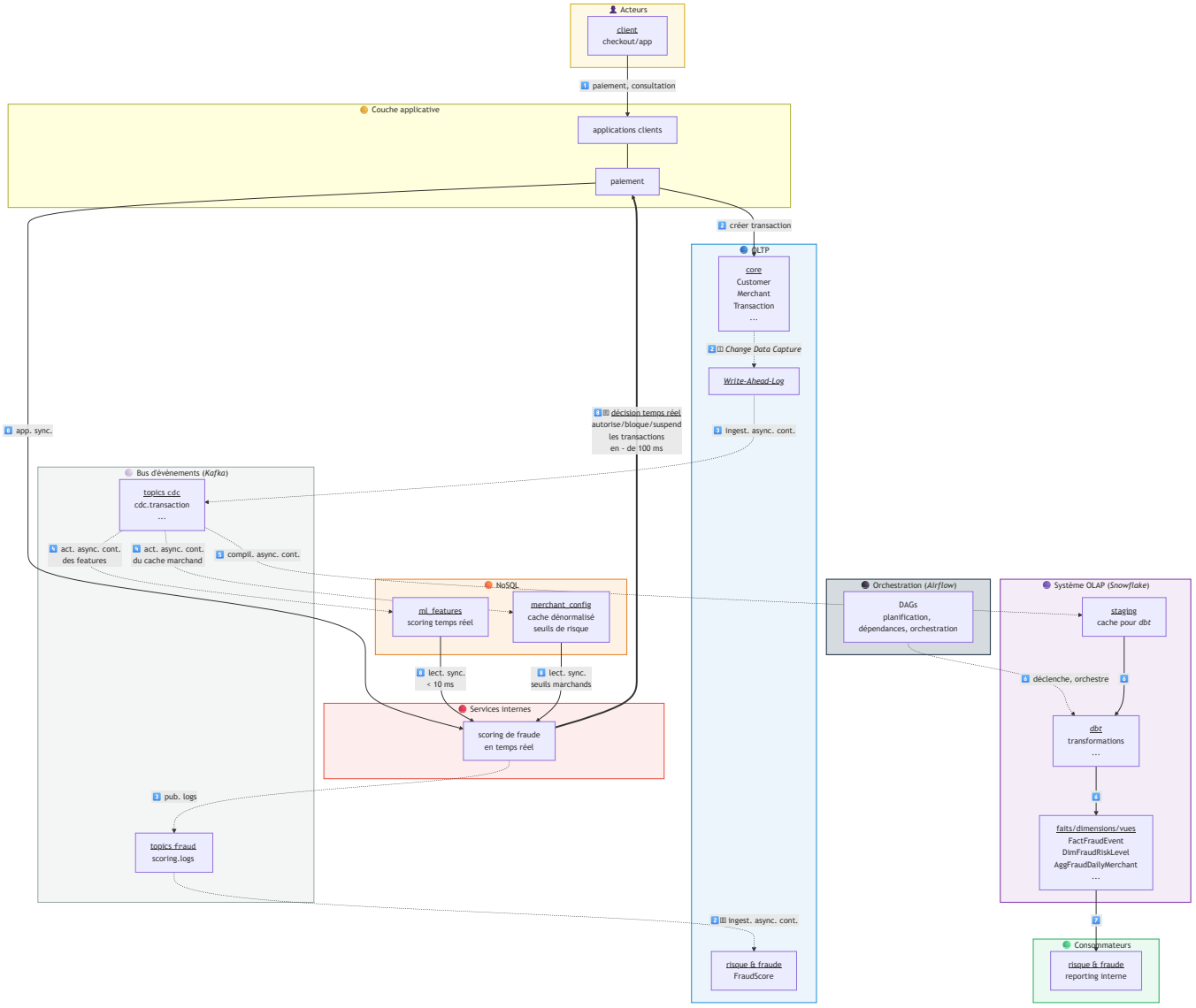
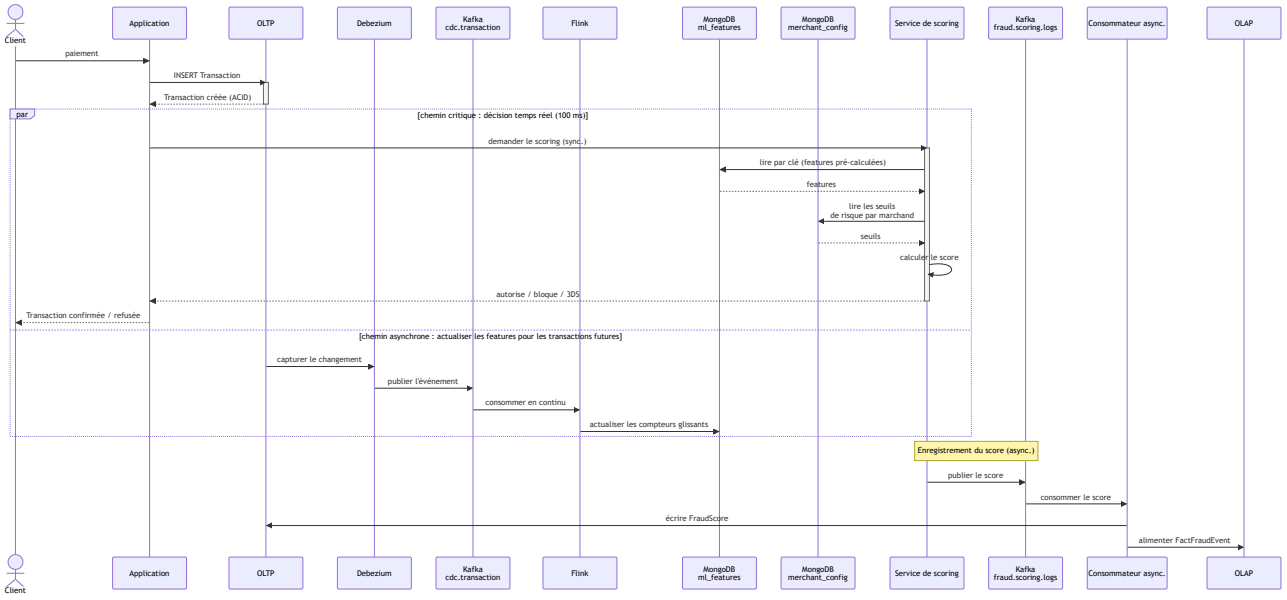



**Diagramme de séquence du scoring de fraude au cours d'une transaction**

Le diagramme de séquence suivant montre comment la fonction de détection de fraude s'insère dans le processus de paiement en temps réel.




## D1.4.2. Flux de données

Explication des flux de données liés à la détection de fraude :

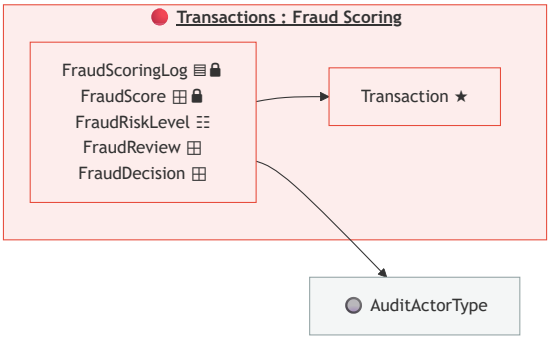
| # | Flux | Mécanisme | Justification |
|---|---|---|---|
| 4 | Topic `cdc` → NoSQL `merchant_config` | act. async. cont. du cache marchands | Cache dénormalisé actualisé à chaque changement de OLTP Core ; évite d'interroger OLTP à chaque événement app |
| 4 | Topic `cdc` → NoSQL `ml_features` | act. async. cont. des features | Le recalcul des features suit le même flux CDC que "NoSQL merchants" ; pas de mécanisme distinct |
| 8 | NoSQL `features` → scoring | lect. sync. < 10 ms | Alimente le modèle de fraude en continu |
| 8 | NoSQL `merchants` → scoring | lect. sync. | Seuils de risque par marchand, consultés en même temps que les features |
| 8R | scoring → Apps | rép. sync. | Autorise/bloque/suspend la transaction ; latence bout en bout < 100 ms ; la décision revient à la couche applicative ; la persistance du score est un flux séparé |
| 3 | scoring → Topic `fraud` | pub. async. | Seul flux qui revient vers OLTP par un mécanisme asynchrone ([D5, §D5.6.4]), distinct de la décision de fraude elle-même |
| 2B | Topic `fraud` → OLTP `FraudScore` | ... | Enregistre le score dans OLTP ; planifié/compilé/orchestré ? |
| 5 | Topic `fraud` → staging OLAP | extract. async. cont. | cf. §D1.3.2. |
| 5 | NoSQL → staging OLAP | transf. plan. | cf. §D1.3.2. |
| 6 | `dbt` + staging → OLAP | transf. planifiée | Alimente `FactFraudEvent` via `dbt` |
| 7 | OLAP → analytiques | req. analytiques | Découplé du chemin transactionnel, pas d'impact sur latence de paiement, quatre consommateurs analytiques distincts, tous alimentés par OLAP ([D1, §D1.2]) : 1 externe: marchand, 3 internes: revenu & produit, risque & fraude, sécurité & conformité |

> *abréviations :*
>
> - act. = actualisation
> - pub. = publication
> - lect. = lecture
> - écr. = écriture
> - cont. = continue
> - sync. = synchrone
> - async. = asynchrone
> - transf. = transformation
> - req. = requêtes
> - rép. = réponse

### D1.4.3. Modèles de données

#### D1.4.3.1. Données transactionnelles

Le diagramme suivant montre une vue compacte du modèle transactionnel logique lié à la détection de fraude. Les détails du modèle transactionnel logique sont dans [D2] (cf. §D26 : *Vue Transactions*) .

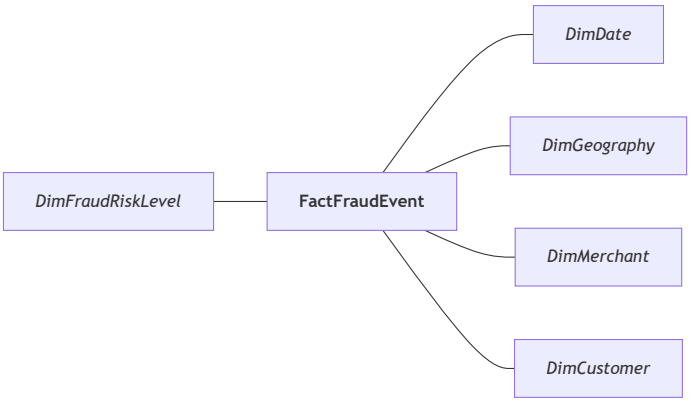



> *Etiquettes de table :*
>
> - ★ : Entité métier principale ;

- ⊞ : Entité métier secondaire (extension directe ou indirecte d'une entité principale) ;
- ☷ : Enumération ;
- ▤ : Journal ;
- 🔒 : Entité immuable (ne peut être ni modifiée ni supprimée après création) ;

#### D1.4.3.2. Données analytiques

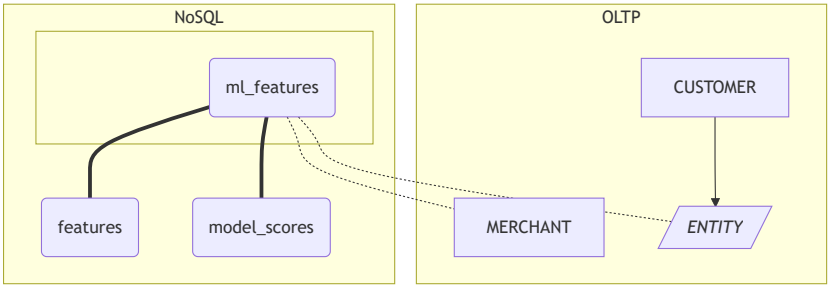
Détails du modèle analytique logique dans [D3] (cf. §D3.3.2 : *Fraudes*).



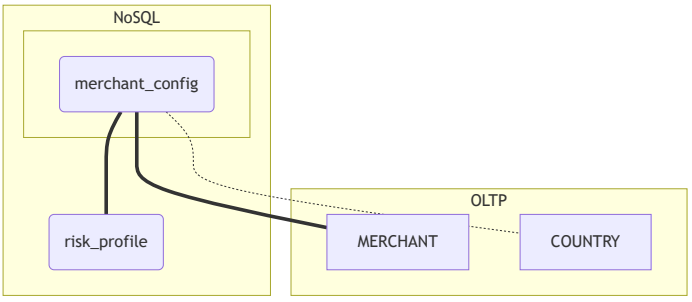

#### D1.4.3.3. Données non-structurées

Détails du modèle des données non-structurées dans [D4] (cf. §D4.5.4 : *ml_features*, §D4.5.6 : *merchant_config*).

**ml_features**




**merchant_config**



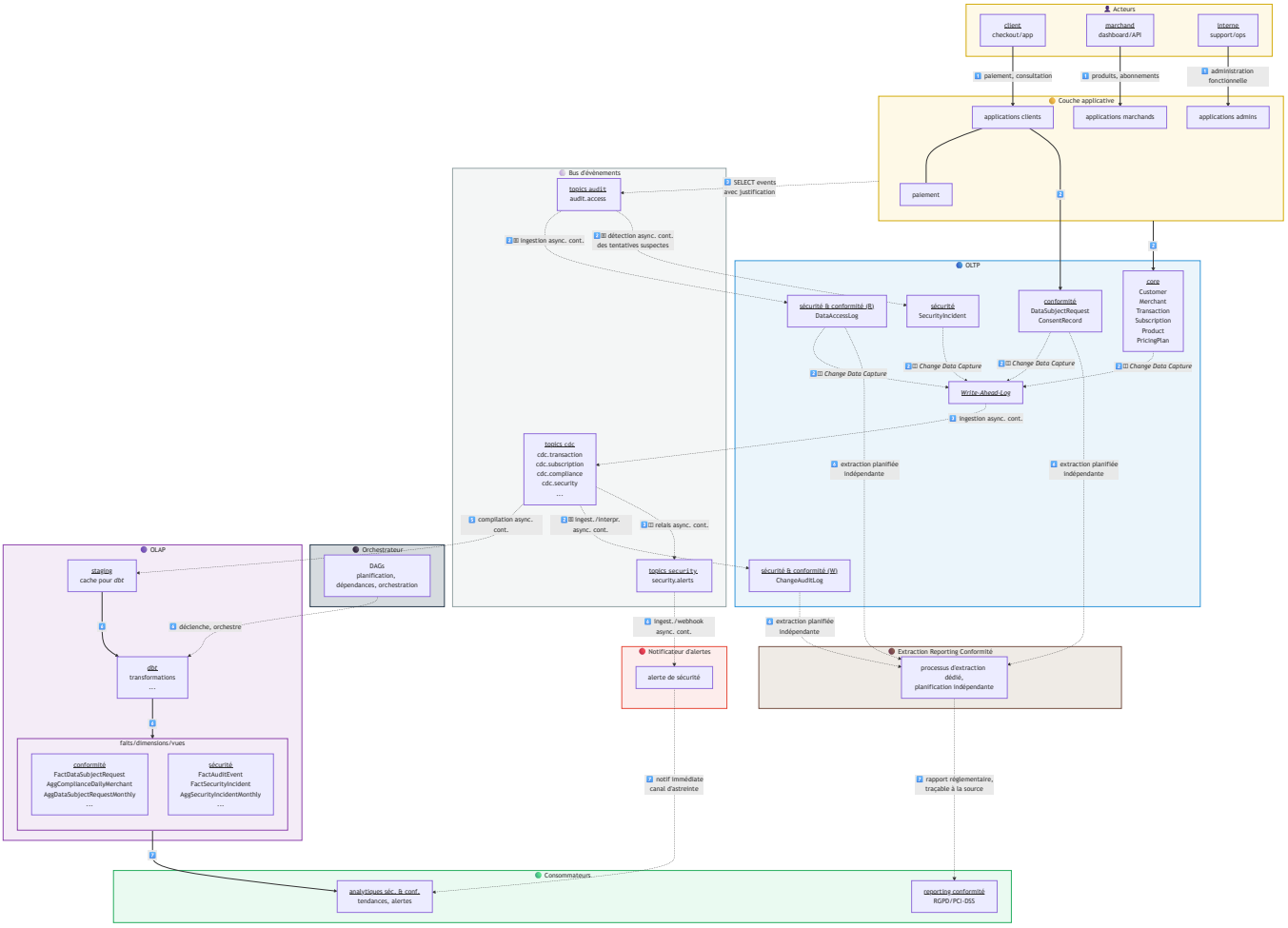

## D1.5. Vue du reporting de sécurité et de conformité

### D1.5.1. Architecture

Le diagramme suivant montre comment le système de données couvre certaines questions de sécurité et de sécurité en intégrant les processus :

- **Sécurité** : journalisation des accès et des écritures, alerting des incidents de sécurité (accès anormaux) en temps réel, analyses a posteriori ;
- **Conformité** : gestion des consentements et des demandes de consultation, de modification ou de suppression (cf. *Data Subject Requests*), reporting réglementaire, analyses a posteriori.

Les flux de données sont détaillés dans §D1.5.2.




### D1.5.2. Flux de données

Explication des flux de données liés à la sécurité et à la conformité :

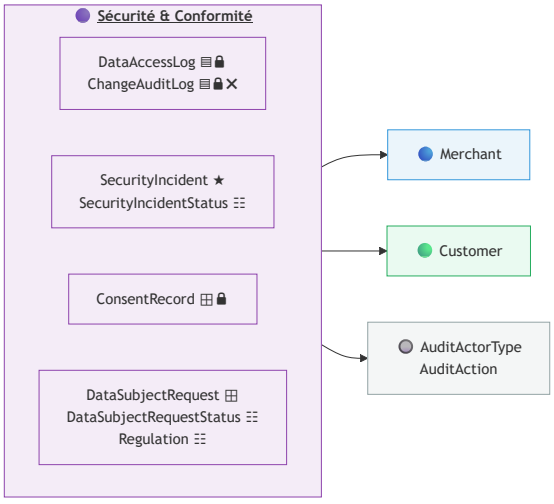
| # | Flux | Mécanisme | Justification |
|---|---|---|---|
| 3 | Apps → Topic `audit` | pub. async. d'événement justifié | `SELECT` ne laisse pas de trace dans *WAL* ; l'application connaît le contexte de l'accès |
| 2B | Topic `audit` → OLTP `DataAccessLog` | ingest. async. cont. | Mappage direct : la justification de l'accès voyage dans le message publié par Apps |
| 3 | OLTP `DataAccessLog` → Topic `cdc` | lect. async. cont. du *WAL* | Réplication standard |
| 3 | OLTP Sécurité → Topic `cdc` | | |
| 3 | OLTP Conformité → Topic `cdc` | | |
| 2B | Topic `cdc` → OLTP `ChangeAuditLog` | ingest. async. cont. | Dérive une ligne d'audit depuis l'événement CDC ; *`ChangeAuditLog` ne doit pas être recapturée par CDC* |
| 2B | Topic `audit` → OLTP `SecurityIncident` | détect. async. cont. | Détection à état (fenêtres glissantes) |
| 3i | Topic `cdc` → Topic `security` | relais async. cont. | Réutilise le fait constaté (l'écriture de `SecurityIncident`) plutôt que de redétecter une seconde fois depuis `audit.access` |
| 6+7 | Topic `security` → équipe sécurité | ... | Le mécanisme de notification (canal d'astreinte) reste indépendant du mécanisme de détection |
| 2 | Apps → OLTP `DataSubjectRequest` | écr. sync. directe | Le client exerce un droit ou donne/retire son consentement via une application |
| | Apps → OLTP `ConsentRecord` | " | " |
| 5 | Topic `cdc` → staging OLAP | extract. async. cont. | cf. §D1.3.2. |
| 6 | `dbt` + staging → OLAP | transf. plan. | cf. §D1.3.2. |
| 7 | OLAP → analytiques | req. analytiques | Découplé du chemin transactionnel, pas d'impact sur latence de paiement, consommateur analyste sécurité & conformité |

| # | Flux | Mécanisme | Justification |
|---|---|---|---|
| 6+7 | OLTP → reporting conformité | extract. plan. indé. | Alimente les rapports RGPD/PCI-DSS sans exposer l'OLTP directement aux outils de reporting ; job d'extraction dédié non intégré au pipeline partagé (robustesse aux pannes du système) |

> *abréviations :*
>
> - lect. = lecture
> - écr. = écriture
> - cont. = continue
> - sync. = synchrone
> - async. = asynchrone
> - pub. = publication
> - ingest. = ingestion
> - détect. = détection
> - transf. = transformation
> - mess. = message
> - req. = requêtes
> - extract. = extraction
> - plan. = planifié
> - indé. = indépendante

### D1.5.3. Modèles de données

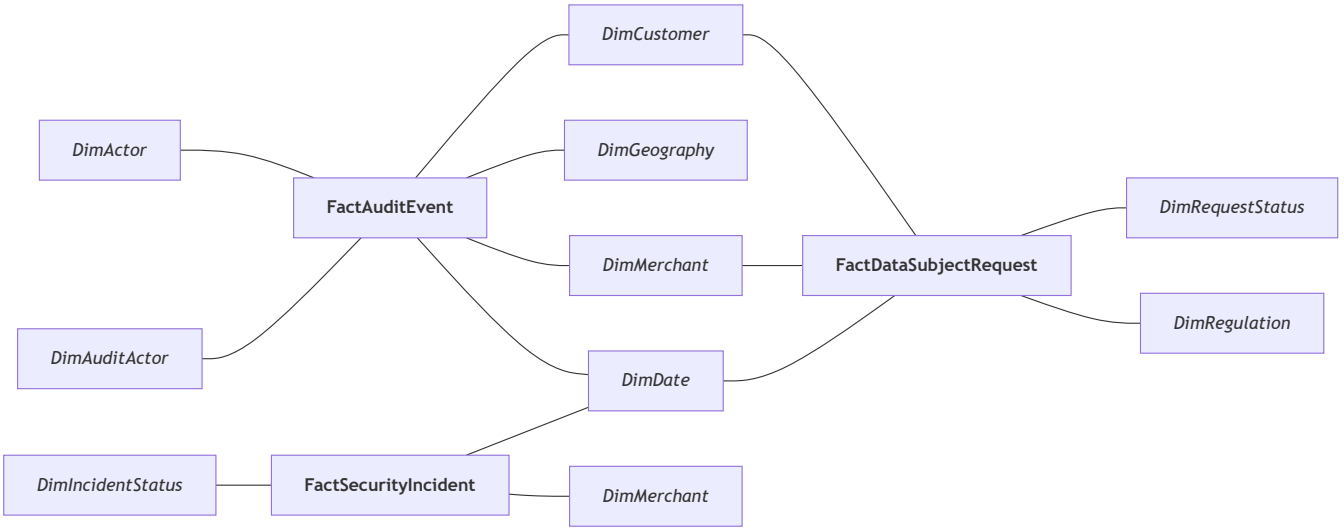
#### D1.5.3.1. Données transactionnelles




> *Etiquettes de table :*
>
> - ★ : Entité métier principale ;
> - ⊞ : Entité métier secondaire (extension directe ou indirecte d'une entité principale) ;
> - ☷ : Enumération ;
> - ▤ : Journal ;
> - 🔒 : Entité immuable (ne peut être ni modifiée ni supprimée après création) ;
> - ✕ : ne peut pas être journalisé (dans les journaux des accès et des écritures) ;

#### D1.5.3.2. Données analytiques

Détails du modèle analytique logique dans [D3] (cf. §D3.3.4 : *Lectures et écritures*, §D3.3.5 : *Demandes de droits*, §D3.3.6 : *Incidents de sécurité*).




#### D1.5.3.3. Données non-structurées

Sans objet : il n'y a pas de collections de données non structurées liées à la sécurité ou à la conformité. Noter que la collection `event_logs` capture les journaux *d'application* et n'a rien à voir avec les accès et les écritures dans la base transactionnelle.

---

## D1.6. Technologies

Détails des technologies retenues dans [D5].

| # | Rôle | Technologies | Remarques | |
|---|---|---|---|---|
| | OLTP | *PostgreSQL* | | |
| | OLAP | *Snowflake* | | |
| | NoSQL | *MongoDB* | orienté documents | |
| | bus d'évènement | *Kafka* | topic streams de topics ; exigé dans [D0] | |
| | CDC (OLTP) | Write-Ahead-Log (WAL) *PostgreSQL* | CDC = Change Data Capture | |
| | CS (NoSQL) | CS *MongoDB* | CS = Change Streams | |
| | Orchestrateur | *Airflow* | | |
| 3 | OLTP CDC > topics `cdc` | connecteur source *Kafka Connect/Debezium* | ingestion asynchrone continue | |
| 4 | topics `app` > NoSQL | connecteur sink *Kafka Connect/MongoDB* | ingestion asynchrone continue | |
| 5 | CS NoSQL > staging OLAP | connecteur *Change Streams* | compilation asynchrone continue | |
| 5 | topics `cdc` > staging OLAP | connecteur sink *Kafka Connect/Snowflake* | extraction asynchrone continue | |
| 5 | topics `fraud` > staging OLAP | connecteur sink *Kafka Connect/Snowflake* | extraction asynchrone continue | |
| 4 | actualisation des features : Topics `cdc` > NoSQL `features` | job *Flink* | actualisation des features, asynchrone continue | |
| 4 | actualisation du cache marchands : Topics `cdc` > NoSQL `merchants` | job *Flink* | actualisation asynchrone continue | |
| 3 🇮 | topics `cdc` > topics `security` | relais *Kafka Streams* | relais asynchrone continue | |
| 2 🇧 | topics `cdc` > OLTP `ChangeAuditLog` | connecteur sink *Kafka Connect* + *SMT (Simple Message Transformation)* | asynchrone continue | |
| 2 🇧 | topics `fraud` > OLTP `FraudScore` | connecteur sink *Kafka Connect* | asynchrone continue | |
| 2 🇧 | topics `audit` > OLTP `DataAccessLog` | connecteur sink *Kafka Connect* | asynchrone continue | |
| 2 🇧 | topics `audit` > OLTP `SecurityIncident` | job *Flink* | détection des tentatives suspectes, asynchrone continue | |
| 6 | topics `security` > canal de notification | connecteur sink *Kafka Connect/webhook* | ingestion asynchrone continue | |

La sélection de l'entrepôt analytique *Snowflake* est une décision prise dans [D5] et justifiée dans [D5-A], mais représentée ici pour permettre au diagramme d'architecture de montrer les choix techniques les plus importants sans avoir à dépendre du document technique [D5].

## D1.7. Remarques

**`event_logs` vs. `audit.access`**

`event_logs` est un journal d'erreur technique ou opérationnelle : une panne de service, un timeout, une exception applicative. `audit.access` trace les accès en lecture des données OLTP.

| | `event_logs` | `audit.access` |
|---|---|---|
| Question | "Qu'est-ce qui a mal fonctionné techniquement?" | "Qui a consulté quelle donnée?" |
| Déclencheur | Erreur/exception d'un service | Lecture réussie ou refusée |
| Exemple | "Card authorization timeout" sur `payment-gateway` | Un agent support consulte le profil d'un client |
| Alimente | Rien % sécurité/conformité | `DataAccessLog` (OLTP), alerting *Flink* ([D5, §D5.9.5]) |

**Reporting Sécurité & Conformité OLTP vs. OLAP**

On a deux types de reporting lié à la *sécurité et à la conformité* :

| | **Reporting réglementaire** | **Reporting analytique** |
|---|---|---|
| Consommateur | Délégué à la protection des données (DPO), responsable conformité | Analyste risque, équipe GRC (*Governance, Risk, Compliance*) |
| Produit | Dépôts légaux, notification de violation, réponse à une demande RGPD | Tableaux de bord de tendance, corrélations, revues périodiques |
| Contrainte | Délai légal opposable (72h, 30/45j), valeur probante | Aucune échéance légale, exploration libre |
| Fraîcheur | Nécessaire à l'instant du dépôt | Suffisante avec un décalage batch |
| Valeur | | Corrélation avec d'autres dimensions (marchand, segment, temps) |
| Dépendance | Traçable à la source, isolé du pipeline partagé | Dépend du pipeline OLAP (Kafka/Airflow/dbt) |
| Source | OLTP | OLAP |

Le chemin OLTP est volontairement minimal, aussi proche de la source que possible. D'un autre coté, l'outil de reporting RGPD/PCI-DSS ne peut pas avoir de connexion directe avec la base transactionnelle (cf. §D1.5.2 "alimente les rapports [...] sans exposer l'OLTP directement aux outils de reporting" ) . D'où la nécessité d'un job d'extraction dédié.

---

## Références

### Spécifications

- [D0] Stripe Project Introduction

### Livrables

- [D2] OLTP Entity-Relation Diagrams
- [D3] OLAP System Schema Design
- [D4] NoSQL Data Model
- [D5] Data Pipeline Architecture
- [D6] Security and Compliance Plan
- [D7] Machine Learning Integration Strategy

### Annexes

- [D2-D] *OLTP Replication, Failover and Disaster Recovery Strategy*
- [D5-A] Data Pipeline Supporting Notes